Flaspland: A Testbed for Routing and Scheduling in ASP

Ryan Murphy[1], Amadé Nemes[1], Javier Romero[1], and Torsten Schaub[1]

University of Potsdam, Germany

Abstract. We present *flaspland*, an Answer Set Programming (ASP) testbed for tackling complex routing and scheduling problems within the Flatland railway simulation environment. To establish a solid theoretical foundation, we provide a rigorous formalization of the Flatland environment, detailing its grid-based topology and navigational dynamics. Building upon this, we propose and analyze three distinct graph-based representations to effectively encode the underlying constraints. Crucially, we introduce a twofold, path-driven ASP encoding scheme that decouples spatial routing from temporal scheduling. Our empirical results demonstrate that our approach outperforms off-the-shelf multi-agent pathfinding approaches. By establishing a testbed for real-world railway scheduling, *flaspland* facilitates the evaluation and advancement of declarative approaches to qualitative and quantitative temporal constraints.

1 Introduction

The ability of Answer Set Programming (ASP [9]) to easily express reachability makes it a prime candidate for addressing routing problems, such as multi-agent path finding [4], phylogenetic inference [3], and wire routing [5]. However, this advantage is challenged once routing is combined with scheduling to enforce deadlines and durations, since fine-grained quantitative temporal reasoning cannot be feasibly represented in ASP.

To address this, we propose railway scheduling as a rich and scalable benchmark for exploring routing and scheduling problems in ASP. We specifically advocate for Flatland [7], a grid-based railway simulation environment developed by major European railway operators, which enables scalable experimentation with complex railway scenarios. The core task is to solve collision-free routing and scheduling problems, which are essentially multi-agent pathfinding problems (MAPF; [13]) augmented with quantitative constraints on train arrival and departure times, durations, energy, and speed.

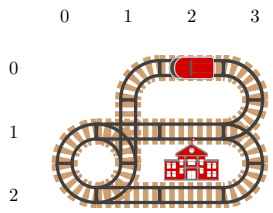


Fig. 1. Visualization of a simple Flatland scenario with one train station and one train

A railway network is modeled as a set of connected cells using matrix notation, as shown in Figure 1. The traversability of cells is defined by specific track types (see Figure 2), which dictate valid transitions. The overall objective is to navigate all trains to their destinations, respecting strict time windows and preventing collisions. Navigation is performed via discrete actions, whose executability is constrained by the train's current direction and the underlying track configuration.

Beyond quantitative scheduling constraints, such railway networks present structural challenges that differentiate it from standard MAPF: (i) Track types strictly limit valid cell transitions, requiring precise modeling of graph connectivity. (ii) A train's ability to move depends on its current orientation, adding another layer of complexity to path planning. (iii) The network topology often features canalized passages with limited switching points, restricting navigational choices and creating bottlenecks.

In what follows, we provide a formal account of Flatland, detailing its environment, invariants, and path definitions. Although Flatland's tasks are intuitively well understood, a rigorous formalization is necessary to establish the correspondence between railway grids and their (graph-based) representations, and ultimately to ensure the correctness and completeness of ASP encodings. We then propose alternative graph-based representations of railway networks and establish their correspondence to the original grid environment. To facilitate empirical evaluation, we introduce the *flaspland* toolkit bridging ASP and the Flatland environment. Finally, we present a novel two-fold, path-driven ASP encoding scheme. Our preliminary empirical evaluation shows that our approach offers significant scalability improvements over off-the-shelf MAPF approaches.

2 Flatland formalization

To begin with, we present some crosscutting concepts. Flatland considers a set $D = \{n, e, s, w\}$ of cardinal *directions* and a set T of *track types*. We define the following operations on directions to compute their opposite and 90-degree rotations:

1. $\overline{(\cdot)} : D \to D$ where $\overline{(\cdot)} = \{\overline{n} \mapsto s, \overline{s} \mapsto n, \overline{e} \mapsto w, \overline{w} \mapsto e\}$
2. $(\cdot)^r : D \to D$ where $(\cdot)^r = \{n^r \mapsto e, e^r \mapsto s, s^r \mapsto w, w^r \mapsto n\}$
3. $(\cdot)^l : D \to D$ where $(\cdot)^l = \{n^l \mapsto w, w^l \mapsto s, s^l \mapsto e, e^l \mapsto n\}$

Clearly, we have $\overline{\overline{d}} = d$, $d^{2r} = d^{2l} = \overline{d}$, and $d^{4r} = d^{4l} = d$ for $d \in D$.



Fig. 2. Basic track types

Flatland's general track types are shown in Figure 2.[1] Type 0 is reserved for empty cells. Specific track types are derived by rotating these general types in 90-degree increments. For example, the simple switch at position $(2, 1)$ in Figure 1 corresponds to Type 2 rotated by $90°$, denoted as $(2, 90)$. Each track type $t \in T$ provides a set of *transitions*, captured by the relation $H_t \subseteq D \times D$ satisfying for all $d, d' \in D$:

1. $(d, d') \in H_t$ iff $(\overline{d'}, \overline{d}) \in H_t$
2. $(d, \overline{d}) \notin H_t$
3. $|\{(d, d'') \in H_t \mid d'' \in D\}| \leq 2$ and $|\{(d'', d) \in H_t \mid d'' \in D\}| \leq 2$

[1] We omit track type 7, the dead end, as recent versions of Flatland no longer produce environments with this type of track.

That is, every transition has a complementary one, no transition represents an immediate about-face, and there may be no more than two transitions for the same incoming and outgoing direction. For instance, we have $H_{(2,90)} = \{(e,e),(w,w),(e,n),(s,w)\}$.

Notation. The directions D, track types T, and transitions $(H_t)_{t\in T}$ are constant across all Flatland environments and are therefore assumed fixed in the remainder of this work.

Grids. In Flatland, a railway network is an $n \times m$ grid of cells C. We associate each *cell* $c \in C$ with a coordinate position and a track type, via the total functions:

1. $p : C \to [0,n] \times [0,m]$
2. $t : C \to T$

We require p to be a bijective function. For the cell[2] $c_{2,1}$ with the simple switch, we have $p(c_{2,1}) = (2,1)$ and $t(c_{2,1}) = (2,90)$. The relation $H_{t(c)}$ provides the set of transitions for a cell $c \in C$, and we call c *trivial* whenever $H_{t(c)} = \emptyset$.

The cell adjacent to cell $c \in C$ in direction $d \in D$ is defined by the partial function

3. $a : C \times D \to C$

such that

$$p(a(c,d)) = \begin{cases} (i-1,j) & \text{if } p(c) = (i,j),\ d = n, \text{ and } (i,j),(i-1,j) \in [0,n] \times [0,m] \\ (i,j+1) & \text{if } p(c) = (i,j),\ d = e, \text{ and } (i,j),(i,j+1) \in [0,n] \times [0,m] \\ (i+1,j) & \text{if } p(c) = (i,j),\ d = s, \text{ and } (i,j),(i+1,j) \in [0,n] \times [0,m] \\ (i,j-1) & \text{if } p(c) = (i,j),\ d = w, \text{ and } (i,j),(i,j-1) \in [0,n] \times [0,m] \end{cases}$$

Function a is partial because of border cells. If $a(c,d) \neq a(c,d')$ then $d \neq d'$ for all $c \in C$ and $d, d' \in D$. Moreover, we have $a(c,d) = c'$ iff $a(c', \overline{d}) = c$ for all $c, c' \in C$ and $d \in D$. For instance, we have $a(c_{2,1}, w) = c_{2,0}$ and $a(c_{2,0}, e) = c_{2,1}$, whereas $a(c_{2,1}, s)$ is undefined.

A non-trivial cell $c \in C$ is connected to another non-trivial cell $c' \in C$ in direction $d \in D$, written as $c \overset{d}{\hookrightarrow} c'$, when the following conditions hold:

1. $a(c,d) = c'$
2. $(d', d) \in H_{t(c)}$ for some $d' \in \{d, d^l, d^r\}$
3. $(d, d'') \in H_{t(c')}$ for some $d'' \in \{d, d^l, d^r\}$

Condition 3 is equivalent to $(d'', \overline{d}) \in H_{t(c')}$. Moreover, we have $c \overset{d}{\hookrightarrow} c'$ iff $c' \overset{\overline{d}}{\hookrightarrow} c$ for all $c, c' \in C$ and $d \in D$. Continuing our example, we get $c_{2,1} \overset{w}{\hookrightarrow} c_{2,0}$ since $a(c_{2,1}, w) = c_{2,0}$, $(s,w) \in H(t(c_{2,1}))$, and $(w,n) \in H(t(c_{2,0}))$, and hence $c_{2,0} \overset{e}{\hookrightarrow} c_{2,1}$, too.

A *transit pair* (c,d) combines a non-trivial cell $c \in C$ and direction $d \in D$ such that $(d', d) \in H_{t(c)}$ for some $d' \in D$. The cell $c_{2,1}$ gives rise to three transit pairs: $(c_{2,1}, e)$, $(c_{2,1}, w)$, and $(c_{2,1}, n)$, corresponding to the three possible outgoing directions. We denote the set of all transit pairs for a grid of cells C by $\mathcal{T}(C)$. We require for each transit pair (c,d) that $c \overset{d}{\hookrightarrow} c'$ for some $c' \in C$. This condition ensures that each outgoing track leads to an incoming track on the neighboring cell.

We define a *transit path* as a sequence $((c_i, d_i))_{i=0}^{n}$ of distinct transit pairs with $n > 0$ such that for all $0 \leq i < n$:

[2] The subscript in $c_{2,1}$ is just to ease readability.

1. $c_i \overset{d_i}{\hookrightarrow} c_{i+1}$
2. $(d_i, d_{i+1}) \in H_{t(c_{i+1})}$

Transit paths are non-trivial, as they comprise at least two transit pairs. We call $((c_i, d_i))_{i=0}^{n}$ a transit path between (c_0, d_0) and (c_n, d_n). Condition 2 can be seen as an instantiation of Condition 3 in the definition of connectedness with the outgoing direction of the subsequent cell. While a cell may be revisited, each visit must be associated with a different outgoing direction. Also, all but the last direction capture transitions between cells; the final one is an arbitrary choice among all outgoing directions of the last cell.

For illustration, consider the following transit path between $c_{0,1}$ and $c_{1,2}$ in Figure 1:

$$((c_{0,1}, s), (c_{1,1}, s), (c_{2,1}, w), (c_{2,0}, n), (c_{1,0}, e), (c_{1,1}, e), (c_{1,2}, e)) \tag{1}$$

Note that cell $c_{1,1}$ is traversed twice, once southwards and once eastwards.

In practice, Flatland environments are always strongly connected. To capture this formally, we require that for any two cells $c, c' \in C$, there is a transit path connecting some transit pairs (c, d) and (c', d') with $d, d' \in D$.

Notation. We associate a Flatland grid with its set C of cells and leave functions p, t, and a implicit.

Trains. Consider a set Z of *trains*, where each train's start and goal positions, initial orientation, earliest departure, and latest arrival are given by the functions:

1. $s : Z \to C$ and $g : Z \to C$ such that $s(z) \neq g(z)$ for all $z \in Z$
2. $o : Z \to D$ such that $(d, o(z)) \in H_{s(z)}$ for some $d \in D$ and for all $z \in Z$
3. $\delta : Z \to \mathbb{N}$ and $\alpha : Z \to \mathbb{N}$ such that $\alpha(z) > \delta(z)$ for all $z \in Z$

The provisos stipulate that the starting and goal positions differ for each train, that the initial orientation of a train is always among the outbound directions of its starting track, and that time is discrete, with the arrival time strictly exceeding the departure time.

Notation. The functions s, g, o, δ, and α associated with the trains in Z are left implicit.

A *Flatland scenario* is a pair (C, Z), comprising a set C of cells and a set Z of trains. A train drive is a transit path that tolerates repeated transit pairs to mimic waiting, meets the train's initial orientation and its start and goal position, and respects the earliest departure time. Accordingly, we define a *train drive* for a train $z \in Z$ and some $m, n \in \mathbb{N}$ in a Flatland scenario (C, Z) as a sequence $((c_i, d_i))_{i=m}^{n}$ of transit pairs such that

1. for all $m \leq i < n$ either
   (a) $c_i \overset{d_i}{\hookrightarrow} c_{i+1}$ and $(d_i, d_{i+1}) \in H_{t(c_{i+1})}$, or
   (b) $(c_i, d_i) = (c_{i+1}, d_{i+1})$
2. $(c_m, d_m) = (s(z), o(z))$
3. $c_n = g(z)$ and $c_i \neq g(z)$ for all $m \leq i < n$
4. $m \geq \delta(z)$

Numbers m and n reflect the actual departure and arrival times of train z. The scheduled arrival time, $\alpha(z)$, is considered separately in delay calculations. Train drives are non-trivial since $s(z) \neq g(z)$.

Actions. Trains have four possible actions:[3] move forward (f), turn left (l), turn right (r), and wait (w). A transit pair (c, d) and one of its transitions $(d^i, d) \in H_{t(c)}$ induce the action a where

1. $a = f$ if
   (a) $H_{t(c)} \cap (\{d^i\} \times D) = \{(d^i, d)\}$ or
   (b) $H_{t(c)} \supset \{(d, d), (d, d')\}$ for $d^i = d$ and some $d' \in \{d^l, d^r\}$
2. $a = l$ if $H_{t(c)} \supset \{(d^r, d), (d^r, d')\}$ for $d^i = d^r$ and some $d' \in \{\overline{d}, d^r\}$
3. $a = r$ if $H_{t(c)} \supset \{(d^l, d), (d^l, d')\}$ for $d^i = d^l$ and some $d' \in \{\overline{d}, d^l\}$

No navigational choice is required in (1a) since there is only one alternative when entering from direction d^i. A second analogous entry point may exist (e.g., track types 6 and 8), but the choice of action f remains unambiguous. The three other cases, (1b), (2) and (3), are analogous in that they all require choosing d as the outgoing direction from two possible alternatives, using action f, l or r, respectively. Transit pairs induce unique actions, either f, l, or r. Two identical transit pairs form a w[ait] action.

Given a train drive $((c_i, d_i))_{i=m}^{n}$ for train $z \in Z$ in a Flatland scenario (C, Z), we define a *plan* for train z in (C, Z) as a sequence $(a_i)_{i=m}^{n-1}$ of actions such that for $m \leq i < n$

1. $a_i = w$ if $(c_i, d_i) = (c_{i+1}, d_{i+1})$ and
2. a_i is induced by (c_i, d_i) otherwise.

Table 1 details a sample plan for train z traveling from $s(z) = c_{0,2}$ to $g(z) = c_{2,2}$ (in Figure 1) given initial conditions $o(z) = w$ and $\delta(z) = 0$. The plan requires only two navigational decisions, at steps 6 and 9, with all other movements being deterministic f actions, except for the wait at step 7. The decision at step 6 is made at cell $c_{1,1}$, where the train has two possible transits: one continuing east, via (e, e), and another turning south, via (e, s), both of which are members of $H_{t(c_{1,1})}$. The choice of outbound direction e is reflected by the inclusion of $(c_{1,1}, e)$ in the train drive. Similarly, when transiting $c_{1,3}$ eastwards at step 9, the train may go north or south, since both (e, n) and (e, s) belong to $H_{t(c_{1,3})}$. The transit pair $(c_{1,3}, s)$ reflects the southbound trajectory of the train.

i	a_i	(c_i, d_i)
0	f	$(c_{0,2}, w)$
1	f	$(c_{0,1}, s)$
2	f	$(c_{1,1}, s)$
3	f	$(c_{2,1}, w)$
4	f	$(c_{2,0}, n)$
5	f	$(c_{1,0}, e)$
6	$\boldsymbol{f}$	$(c_{1,1}, e)$
7	w	$(c_{1,2}, e)$
8	f	$(c_{1,2}, e)$
9	$\boldsymbol{r}$	$(c_{1,3}, s)$
10	f	$(c_{2,3}, w)$
11		$(c_{2,2}, w)$

Table 1. A plan from $c_{0,2}$ to the train station in $c_{2,2}$ in Figure 1

Collisions. In Flatland, trains appear upon departure and disappear once they reach their destination. Two distinct drives $\big((c_i, d_i)\big)_{i=m}^{n}$ and $\big((c'_i, d'_i)\big)_{i=m'}^{n'}$ have

1. a *vertex conflict* if $c_i = c'_i$ for some $i \in [\max(m, m'), \min(n, n')]$
2. an *edge conflict* if $c_{i+1} = c'_i$ and $c_i = c'_{i+1}$ for some $i, i{+}1 \in [\max(m, m'), \min(n, n')]$

Solution. A *solution* to a Flatland scenario (C, Z) is a family $(\pi_z)_{z \in Z}$ of plans π_z such that all drives underlying $(\pi_z)_{z \in Z}$ are pairwisely vertex and edge conflict-free. The delay of a plan π_z for a train $z \in Z$ is defined as $delay(\pi_z) = \max(0, n - \alpha(z))$ where n is

[3] Flatland provides a fifth action, *do nothing*, which we consider superfluous and therefore omit from this formalization.

the actual arrival time of z with plan π_z. A *delay optimal* solution to (C, Z) is a solution $(\pi_z)_{z \in Z}$ to (C, Z) with a minimum value of $\sum_{z \in Z} delay(\pi_z)$

3 Graph representations

We now consider alternative graph representations for modeling railway networks in Flatland. In particular, we must account for the complex movement dynamics around switches. The adjacency of two cells is a necessary condition for a transition, but not a sufficient one. Furthermore, the incoming direction of the train determines its valid outbound movements. For instance, cell $c_{2,1}$ has three neighboring cells in Figure 1. However, only a train facing east can transit from $c_{2,1}$ to its adjacent cells $c_{1,1}$ or $c_{2,2}$, and its neighboring cell $c_{2,0}$ can only be reached by trains facing south or west. In what follows, we translate grid environments into alternative graph structures that inherently encode these directional constraints. We establish one-to-one correspondences between transit paths in Flatland and paths within the respective graphs.

Graphs with edge transition functions. Given a grid C, we define the directed graph $\mathcal{A}(C) = (V, E)$ as

1. $E = \bigcup_{c \in C, d \in D} \{(c, c') \mid c \xhookrightarrow{d} c'\}$
2. $V = \{c, c' \mid (c, c') \in E\}$

The graph $\mathcal{A}(C)$ contains no trivial cells from C. To capture transitions, we associate with each graph $\mathcal{A}(C) = (V, E)$ two partial functions on its set of edges:

1. $d : E \to D$ where $d((c, c')) = d$ if $c \xhookrightarrow{d} c'$ for $(c, c') \in E$
2. $p : E \to 2^D$ where $p((c, c')) = \{d'' \mid (d'', d) \in H_{t(c)}\}$ if $c \xhookrightarrow{d} c'$ for $(c, c') \in E$.

Function d reflects the direction of a directed edge, while p gives the necessary arriving directions when leaving a node. For illustration, consider the application of function d and p to the incoming and outgoing edges of node $c_{2,1}$ from Figure 2.

$$d((c_{2,0}, c_{2,1})) = e \qquad p((c_{2,1}, c_{2,0})) = \{s, w\} \tag{2}$$

$$d((c_{1,1}, c_{2,1})) = s \qquad p((c_{2,1}, c_{2,2})) = \{e\} \tag{3}$$

$$d((c_{2,2}, c_{2,1})) = w \qquad p((c_{2,1}, c_{1,1})) = \{e\} \tag{4}$$

The interplay of both functions determines whether an incoming and outgoing edge form a transition over a node. This is the case for a node c, whenever $d((c', c)) \in p((c, c''))$ for some connected nodes c', c''. Four combinations of the mappings above satisfy this condition for $c_{2,1}$, which reflect the four possible transitions over $c_{2,1}$.

Given a grid C, we define a path in $\mathcal{A}(C)$ as a sequence $(c_i)_{i=0}^n$ of vertices with $n > 0$ such that

1. $(c_i, c_{i+1}) \in E$ for all $0 \leq i < n$
2. $d((c_{i-1}, c_i)) \in p((c_i, c_{i+1}))$ for all $0 < i < n$
3. if $c_i = c_j$ then $c_{i+1} \neq c_{j+1}$ for all $0 \leq i < j < n$

We call $(c_i)_{i=0}^n$ a path between c_0 and c_n in $\mathcal{A}(C)$. For example, the transit path $((c_{1,1}, s), (c_{2,1}, w), (c_{2,0}, n))$ between $c_{1,1}$ and $c_{2,0}$ in our grid C in Figure 1 corresponds to the path $(c_{1,1}, c_{2,1}, c_{2,0})$ in $\mathcal{A}(C)$ and satisfies $d((c_{1,1}, c_{2,1})) \in p((c_{2,1}, c_{2,0}))$.

The following proposition shows that our graph representation preserves all paths in the original grid.

Proposition 1. *Let C be a grid.*

1. *If $((c_i, d_i))_{i=0}^n$ is a transit path in grid C, then $(c_i)_{i=0}^n$ is a path in the graph $\mathcal{A}(C)$.*
2. *If $(c_i)_{i=0}^n$ is a path in the graph $\mathcal{A}(C)$, then $((c_i, d_i))_{i=0}^n$ is a transit path in grid C, where $d_i = d((c_i, c_{i+1}))$ for $0 \leq i < n$ and $d_n = d((c_n, c))$ for some $c \in C$.*

Graph with sub-nodes. Given a grid C, we define the directed graph $\mathcal{B}(C) = (V, E)$ as

1. $V = \mathcal{T}(C)$
2. $E = \{((c, d), (c', d')) \in V \times V \mid c \overset{d}{\hookrightarrow} c', (d, d') \in H_{t(c')}\}$

The graph $\mathcal{B}(C)$ connects all transit pairs of a grid. Fragmenting cells into transit pairs and relating them in terms of their connectivity allows us to account for the transitions over the various track types.

The cell $c_{2,1}$ is now decomposed into three nodes, $(c_{2,1}, e)$, $(c_{2,1}, w)$, and $(c_{2,1}, n)$, corresponding to its three transit pairs. Intuitively, the above edge transition functions are compiled into the refined graph structure. Each node (or transit pair) represents one way of exiting a node. The four possible passages over $c_{2,1}$ are reflected by the four edges entering the three nodes derived from $c_{2,1}$.

Given a grid C, we define a path in $\mathcal{B}(C)$ as a sequence $((c_i, d_i))_{i=0}^n$ of transit pairs with $n > 0$ such that

1. $((c_i, d_i), (c_{i+1}, d_{i+1})) \in E$ for all $0 \leq i < n$
2. if $(c_i, d_i) = (c_j, d_j)$, then $i = j$ for all $0 \leq i < j < n$

We call $((c_i, d_i))_{i=0}^n$ a path between (c_0, d_0) and (c_n, d_n) in $\mathcal{B}(C)$. In fact, the transit path $((c_{1,1}, s), (c_{2,1}, w), (c_{2,0}, n))$ in our grid C in Figure 1 is also a path in $\mathcal{B}(C)$.

Proposition 2. *Let C be a grid.*

1. *If $((c_i, d_i))_{i=0}^n$ is a transit path in C, then $((c_i, d_i))_{i=0}^n$ is a path in the graph $\mathcal{B}(C)$.*
2. *If $((c_i, d_i))_{i=0}^n$ is a path in the graph $\mathcal{B}(C)$, then $((c_i, d_i))_{i=0}^n$ is a transit path in C.*

Graphs with hyper-edges. Given a grid C, we define the graph $\mathcal{H}(C) = (V, E)$ as

1. $E = \bigcup_{c \in C, d \in D} \{(c, c', c'') \mid c \overset{d}{\hookrightarrow} c', c' \overset{d'}{\hookrightarrow} c'', (d, d') \in H_{t(c')}\}$
2. $V = \{c, c', c'' \mid (c, c', c'') \in E\}$

The directed graph $\mathcal{H}(C)$ is using directed hyper-edges among the non-trivial cells of the original grid C. Each hyper-edge captures a valid passage over a node by connecting it to its predecessor and successor in this transition.

We get four hyper-edges centered upon node $c_{2,1}$, namely,

$$(c_{2,0}, c_{2,1}, c_{2,2}) \qquad (c_{2,0}, c_{2,1}, c_{1,1}) \tag{5}$$

$$(c_{1,1}, c_{2,1}, c_{2,0}) \qquad (c_{2,2}, c_{2,1}, c_{2,0}) \tag{6}$$

These hyper-edges correspond to the four passages over $c_{2,1}$ discussed above.

Given a grid C, we define a path in $\mathcal{H}(C)$ as a sequence $(c_i)_{i=0}^n$ of vertices with $n > 0$ such that

1. $(c_{i-1}, c_i, c_{i+1}) \in E$ for all $0 < i < n$
2. $(c, c_0, c_1) \in E$ and $(c_{n-1}, c_n, c') \in E$ for some $c, c' \in V$
3. if $c_i = c_j$ then $c_{i+1} \neq c_{j+1}$ for all $0 \leq i < j < n$

We call $(c_i)_{i=0}^n$ a path between c_0 and c_n in $\mathcal{H}(C)$. Condition 2 ensures that the constraints imposed by hyper-edges are satisfied at the endpoints of each path, and for paths of length two. For example, the transit path $((c_{1,1}, s), (c_{2,1}, w), (c_{2,0}, n))$ between $c_{1,1}$ and $c_{2,0}$ in our grid C corresponds to the path $(c_{1,1}, c_{2,1}, c_{2,0})$ in $\mathcal{H}(C) = (V, E)$ satisfying $\{(c_{0,1}, c_{1,1}, c_{2,1}), (c_{1,1}, c_{2,1}, c_{2,0}), (c_{2,1}, c_{2,0}, c_{1,0})\} \subseteq E$.

Proposition 3. *Let C be a grid.*

1. *If $((c_i, d_i))_{i=0}^n$ is a transit path in grid C, then $(c_i)_{i=0}^n$ is a path in the graph $\mathcal{H}(C)$.*
2. *If $(c_i)_{i=0}^n$ is a path in the graph $\mathcal{H}(C)$, then $((c_i, d_i))_{i=0}^n$ is a transit path in grid C, where $c_i \overset{d_i}{\hookrightarrow} c_{i+1}$ for $0 \leq i < n$ and $c_n \overset{d_n}{\hookrightarrow} c$ for some $c \in C$.*

4 *Flaspland*: Flatland to ASP and back

Flaspland is an open-source toolkit connecting ASP with the Flatland ecosystem. It converts Flatland environments into a standard ASP fact format, and maps the resulting answer sets back to Flatland actions for execution and visualization. By integrating directly with Flatland's native generator, *flaspland* bypasses manual instance construction and supports batch generation to streamline large-scale experiments. Additionally, the toolkit serves as a benchmark repository for reproducible evaluation, housing the 150 environments used in the Flatland 3 competition finals. These instances vary in grid size, agent count, city distribution, train speeds, and malfunction rates. The *flaspland* toolkit is publicly available at [11].

5 Encodings

We now present the ASP encodings for addressing routing and scheduling in Flatland, starting with the representation of the environment's basic topological and navigational concepts. All encodings are available at [12].

The fact format of the basic concepts regarding directions is given in Listing 1.1.

```
1      dir(n).          dir(s).         dir(w).          dir(e).
2  inverse(n,s).   inverse(s,n).  inverse(w,e).   inverse(e,w).
3     left(n,w).       left(s,e).      left(w,s).       left(e,n).
4    right(n,e).      right(s,w).     right(w,n).      right(e,s).
5   offset(n,-1,0). offset(s,1,0). offset(w,0,-1). offset(e,0,1).
```

Listing 1.1. Representation of directions (`n`,`s`,`w`,`e`)

Predicate `dir/1` gives the cardinal directions, `inverse/2` provides complementary directions, `left/2` and `right/2` capture 90-degree rotations, and `offset/3` gives a coordinate offset when moving in a certain direction.

In Flatland, each track type has a numeric identifier that is based on its possible transitions; it is represented via predicate `transition/2`. Listing 1.2 contains all facts reflecting the four transitions of the simple switch at position $(2, 1)$ in Figure 1.[4]

```
42 transition(3089,(s,w))
43 transition(3089,(w,w))
44 transition(3089,(e,e))
45 transition(3089,(e,n))
```

Listing 1.2. Selected lines of track type representations

The original grid is represented by `cell/2` atoms, where the first argument reflects the coordinates of a cell and the second gives its track type. These coordinates serve as both the cell identifier and the value of the position function p. The available transitions for each cell are obtained by joining the `cell/2` and `transition/2` predicates.

```
1 start(0,(0,2),0,w).
2 end(0,(2,2),12).

4 cell((0,0),    0). cell((0,1),16386). cell((0,2),1025). cell((0,3),4608).
5 cell((1,0),16386). cell((1,1),38433). cell((1,2),1025). cell((1,3),6672).
6 cell((2,0),   72). cell((2,1), 3089). cell((2,2),1025). cell((2,3),2064).
```

Listing 1.3. The fact representation of our running example (Figure 1)

Furthermore, the instance format uses `start/4` and `end/3` atoms to define the initial and target configurations of trains. The first two arguments of these predicates identify the train z and its starting $s(z)$ or ending $g(z)$ position, respectively (e.g. Lines 1 and 2 of Listing 1.3). The third and fourth arguments of `start/4` indicate the earliest departure time $\delta(z)$ and initial orientation $o(z)$, while the third argument of `end/3` specifies the latest arrival time $\alpha(z)$.

The adjacency function a and the connectivity relation $\hookrightarrow$ are defined in Listing 1.4, using predicates `adjacent/3` and `connected/3`, respectively.

```
1 adjacent((X,Y),D,(X',Y')) :-  cell((X,Y),_), cell((X',Y'),_),
2     offset(D,DX,DY), X'=X+DX, Y'=Y+DY.

4 connected((X,Y),D,(X',Y')) :- cell((X,Y),T), cell((X',Y'),T'),
5     adjacent((X,Y),D,(X',Y')),
6     transition(T,(D',D)), transition(T',(D,D'')).
```

Listing 1.4. Representation of grid functions

The fact representation introduced above, generated by our *flaspland* framework from Flatland scenarios, serves as the basic input for our ASP encodings. Building upon this basis, we investigate the efficacy of the three graph-based representations defined in Section 3 in modeling train dynamics and interactions.

Our encoding design is driven by two main objectives. First, we aim to evaluate the computational trade-offs of the proposed topological representations, specifically regarding how the choice of structural abstraction influences ASP solving performance. Second, our encodings must robustly address the spatio-temporal complexities inherent to train scheduling. Foremost is the need to model waiting actions for collision avoidance,

[4] The value 3089 at $(2, 1)$ in Figure 1 derives from the 16-bit encoding `0000 1100 0001 0001` for $H_{(2,1)}$. It consists of four 4-bit blocks representing incoming orientations (`n`,`e`,`s`,`w`), where each bit flags a valid outgoing direction (`n`,`e`,`s`,`w`).

which inherently decouples spatial transitions from strict temporal succession. Further complexities stem from the asynchronous life-cycle of trains, such as their variable appearance, delayed departures, and disappearance upon arrival. Moreover, a train's initial orientation can typically only be verified upon its actual departure.

We address these spatio-temporal complexities by decomposing the synthesis of train drives into two distinct aspects: spatial routing and temporal scheduling. The routing component establishes the backbone of a train's journey by identifying a valid path between its start and destination. Crucially, while these paths may revisit nodes, they prohibit consecutive duplicate nodes. This abstraction of temporal waiting ensures that spatial transitions can be directly verified against the underlying track topology. The scheduling component then refines these spatial paths into fully realized train drives by inserting appropriate waiting times to enforce collision avoidance.

In the following, we present a specialized path encoding for each graph representation and subsequently integrate these encodings into a unified drive encoding. We employ the predicate `path/3` across all settings to represent paths within the respective graphs; the arguments denote the train, the node, and the node's position in the path, respectively.

Encodings based on directed graphs with edge transition functions. We begin by introducing encodings based on a directed graph representation augmented with edge transition functions. The encoding in Listing 1.5 maps a Flatland grid into this structure.

```
1  edge(C,C') :- connected(C,_,C').
2  node(C;C') :- edge(C,C').

4  d(C,C',D) :- connected(C,D,C').
5  p(C,C',D) :- connected(C,D',C'), cell(C,T), transition(T,(D,D')).
```

Listing 1.5. Encoding for directed graphs with edge functions.

The graph topology is captured by the predicates `node/1` and `edge/2`, while `d/3` and `p/3` encode the functions d and p, respectively.

The encoding in Listing 1.6 identifies spatial paths of length up to `h` between each train's starting and goal positions.

```
1  time(0..h).

3  path(Z,C,0; Z,C',1) :- start(Z,C,_,D), adjacent(C,D,C').
4  { path(Z,C',T+1): edge(C,C') } = 1 :-
5        path(Z,C,T), time(T+1), not eop(Z,T).

7  eop(Z,T) :- end(Z,C,_), path(Z,C,T).
8  :- start(Z,_,_,_), not eop(Z,_).

10 :- path(Z,C,T), path(Z,C',T+1), path(Z,C'',T+2),
11    d(C,C',D), not p(C',C'',D).
```

Listing 1.6. Pathfinding with edge-functions

Line 1 defines the available time steps, while Line 3 initializes each train at its designated starting position and orientation. The path is iteratively extended (Line 4-5) until the destination is reached. Termination is signaled by the `eop(Z,T)` atom (Line 7),[5] indicating that train `Z` has arrived at its destination at time step `T`. The integrity constraint

[5] `eop` stands for "end of path".

in Line 8 ensures that every train eventually reaches its goal. Crucially, Line 11 enforces transition validity by applying the edge functions d and p to consecutive nodes in the path, verifying that the incoming and outcoming edge form a valid transition.

Encodings based on directed graphs with sub-nodes. We now introduce encodings that leverage the augmented node structure. The translation from the standard Flatland grid into this representation is provided in Listing 1.7.

```
1  node((X,Y,D)) :- cell((X,Y),T), transition(T,(_,D)).

3  edge((X,Y,D),(X',Y',D')) :- node((X,Y,D)), node((X',Y',D')),
4      connected((X,Y),D,(X',Y')),
5      cell((X',Y'),T'), transition(T',(D,D')).
```

Listing 1.7. Encoding for directed graphs with sub-nodes.

In this setting, each node corresponds to a transit pair (a cell and an outgoing direction), thereby encoding valid transitions directly into the graph's edge construction.

Listing 1.8 adapts the pathfinding logic to this refined structure.

```
1  time(0..h).

3  path'(Z,(X,Y,D),0) :- start(Z,(X,Y),_,D).
4  { path'(Z,C',T+1) : edge(C,C') } = 1 :-
5          path'(Z,C,T), time(T+1), not eop(Z,T).

7  eop(Z,T) :- end(Z,(X,Y),_), path(Z,(X,Y),T).
8  :- start(Z,_,_,_), not eop(Z,_).

10 path(Z,(X,Y),T) :- path'(Z,(X,Y,D),T).
```

Listing 1.8. Pathfinding with sub-nodes

While the rules in Lines 1–8 mirror the logic of the edge-function-based encoding, they operate on transit pairs rather than simple cells. To provide a uniform interface for subsequent scheduling, the internal predicate `path'/3` is projected back onto the cell-based `path/3` in Line 10. In this representation, transition validity is guaranteed by construction and requires no explicit verification during path generation.

Encodings based on directed graphs with hyper-edges. Lastly, we present encodings that utilize hyper-edges. Listing 1.9 provides the translation into this representation.

```
1  hedge(C,C',C'') :- connected(C,D,C'), connected(C',D',C''),
2            cell(C',T), transition(T,(D,D')).

4  node(C;C';C'') :- hedge(C,C',C'').
```

Listing 1.9. Encoding for directed graphs with hyper-edges.

While `node/1` continues to designate cell-based nodes, the valid transitions are modeled via the ternary predicate `hedge/3`. This approach naturally captures the dependencies between a node's entry and exit points.

Listing 1.10 details the corresponding pathfinding encoding.

```
1  time(0..h).

3  path(Z,C,0; Z,C',1) :- start(Z,C,_,D), adjacent(C,D,C').
4  { path(Z,C'',T+1) : hedge(C,C',C'') } = 1 :-
```

```
5       path(Z,C,T-1), path(Z,C',T), time(T+1), not eop(Z,T).

7  eop(Z,T) :- end(Z,C,_), path(Z,C,T).
8  :- start(Z,_,_,_), not eop(Z,_).
```

Listing 1.10. Pathfinding with hyper-edges

The path generation rules (Lines 1–8) leverage the hyper-edges to ensure that every step in the path is valid according to the underlying track topology. This representation is arguably the most direct, as the hyper-edges explicitly encapsulate the ternary nature of cell transitions without requiring auxiliary functions or node fragmentation.

Encodings of train drives. To transform spatial paths into fully realized train drives, we must account for potential waiting times at each node, which are essential for collision avoidance. Analogous to paths, train drives are represented via the `drive/3` predicate. Train waiting is modeled as a partial mapping from drive steps to path steps, captured by the `map/3` predicate, taking the train, drive step, and path step as arguments. For instance, the atoms `map(z,5,3)`, `map(z,6,3)`, `map(z,7,3)` indicate that train `z` occupies its third position in the path from time point `5` to `7`.

```
1    map(Z,D-1, -1)    :- start(Z,_,D,_).
2  { map(Z,D+1,P  ) } :- map(Z,D,P), not eop(Z,P), D<h.
3    map(Z,D+1,P+1)    :- map(Z,D,P), not map(Z,D+1,P),
4                         path(Z,_,P+1), D<h.
5                      :- path(Z,_,P), not map(Z,_,P).

7        drive(Z,C,D) :- path(Z,C,P), map(Z,D,P).
```

Listing 1.11. Assembling train drives from paths

The `map/3` predicate is defined iteratively in Lines 1-5 of Listing 1.11. Given the earliest departure time $\delta(z)$, we initialize the mapping with drive step $\delta(z) - 1$ and path step -1. Subsequent drive steps `D+1` can map to the same path step `P` as `D` (Line 2), or advance to the next path step `P+1` (Lines 3-4). The integrity constraint in Line 5 ensures every path step is matched by at least one drive step. Finally, Line 7 derives the `drive` atoms from the `path` and `map` atoms.

Encodings of collisions, plans, and delay optimality. Beyond individual train dynamics, the feasibility of a solution hinges on the absence of inter-train interference. To this end, the vertex and edge conflict conditions established in Section 2 are implemented in Listing 1.12. These constraints ensure that no two trains occupy the same cell simultaneously and prevent head-on collisions by forbidding trains from traversing the same edge in opposite directions at the same time step.

```
1  :- 2 { drive(Z,C,T) }, cell(C,_), T=1..h.

3  move(C,C',T) :- connected(C,D,C'), drive(Z,C,T-1), drive(Z,C',T).
4  :- move(C,C',T), move(C',C,T), C < C'.
```

Listing 1.12. Vertex and edge conflicts

Finally, we address the optimization of the overall schedule. While the pathfinding component identifies valid spatial routes, the scheduling component must minimize the total deviation from the target arrival times. This is achieved by minimizing the sum of delays across all trains, effectively penalizing any dwelling time that extends a train's

journey beyond its scheduled window. By combining these constraints, the resulting ASP encoding provides a complete framework for synthesizing delay-optimal solutions within the Flatland environment.

```
1  #minimize{ D : delay(Z,T,D), eop(Z,T) }.
```

Listing 1.13. Delay minimization

Finally, we note that the concrete sequences of actions, or plans, are implicitly defined by the underlying train drives. Within the *flaspland* framework, these plans are primarily utilized for visualization purposes and are therefore omitted here for conciseness.

Encodings using difference constraints. A key advantage of our decoupled approach, separating routing from scheduling, is its inherent modularity, which facilitates the integration of alternative scheduling formalisms. Inspired by railway scheduling approaches [1], we formalize the scheduling phase using difference constraints of the form $u - v \leq d$, where u, v are integer variables and d is an integer constant. In *clingo*[DL] [6], they are represented by atoms of the form `&diff{u-v}<=d` (or equivalents). To emulate the temporal-spatial mapping within this framework, each term tuple `(Z,P)` is assigned an integer representing the drive step at which train `Z` arrives at position `P` on its path. For example, the three `map/3` atoms discussed above are captured by assignments such as `(z,3)` $\mapsto 5$, where `(z,3)` is a term and its assignment of 5 is left implicit.

```
1  &diff{ (Z,0  ) } >= D    :- start(Z,_,D,_).
2  &diff{ (Z,P-1) } < (Z,P) :- train(Z), P=1..h.
3  &diff{ (Z,P  ) } <= h    :- end_of_path(Z,P).

5  { resolve((Z,P),(Z',P')) ; resolve((Z',P'),(Z,P)) } = 1 :-
6          path(Z,C,P), path(Z',C,P'), Z < Z'.

8  &diff{ (Z,P+1) } <= (Z',P') :- resolve((Z,P),(Z',P')).

10 :- resolve((Z,P),(Z',P'+1)), resolve((Z',P'),(Z,P+1)), Z < Z'.
```

Listing 1.14. Assembling train drives from paths via difference constraints

These assignment are defined in Lines 1-3. Line 1 ensures the initial path step occurs no earlier than the train's earliest departure time. Line 2 enforces chronological progression by requiring strictly increasing drive steps for all subsequent path steps. Finally, Line 3 ensures that the final path step is reached no later than the overall time horizon.

The remaining rules enforce collision avoidance. When two trains share a cell along their paths, the choice rule in Lines 5-6 resolves potential vertex conflicts by assigning traversal precedence to one of the trains. The difference constraint in Line 8 ensures that the prioritized train reaches its next path step no later than when the second train enters the contested cell. An edge conflict arises when trains `Z` and `Z'` claim mutual precedence over each other's subsequent path steps `P+1` and `P'+1`, respectively. (Line 10). This cycle of mutual precedence is only possible if both trains are assigned the same drive step for their next positions, resulting in an illegal swap. The separate encoding required to derive the final `drive/3` representation is omitted for brevity.

6 Experiments

We evaluate the three proposed topological representations and their hybrid variant against a baseline encoding on 360 benchmark instances to assess computational perfor-

mance and scalability. The baseline adapts a mature, off-the-shelf MAPF encoding [2], augmented with the necessary constraints to handle Flatland's directional dynamics. Our benchmark set derives from the first ten base environments described in Section 4, each consisting of a 30×30 grid populated with seven trains. To assess scalability with agent density, we varied the number of active trains while maintaining fixed track topologies. For each of the ten base environments and subset size $k \in \{2, \ldots, 6\}$, we sampled seven train combinations. Including the unique full instance for $k = 7$ yields $10 \times (5 \times 7 + 1) = 360$ instances in total. For each instance, we set the maximum time horizon to the latest arrival time among its participating trains to ensure temporal feasibility. We ran all instances on a cluster with Intel Xeon E5-2650v4@2.9GHz CPUs and 64GB of RAM running Debian Linux 10. We used *clingo* 5.8 and the corresponding *clingo*[DL] version, imposing a 1500-second timeout and a 15GB memory limit per run.

		MAPF			EDGFUNCS			SUBNODES			HYPEDGES			DIFCONS		
$\|T\|$	$\|I\|$	Grd	Tot	(×)	Grd	Tot	(×)	Grd	Tot	(×)	Grd	Tot	(×)	Grd	Tot	(×)
2	70	3	9	(**0**)	10	16	(**0**)	3	3	(**0**)	9	14	(**0**)	**2**	**2**	(**0**)
3	70	5	81	(**0**)	18	55	(**0**)	**4**	**6**	(**0**)	16	53	(**0**)	6	7	(**0**)
4	70	**6**	484	(8)	25	247	(2)	6	49	(**0**)	23	243	(1)	13	**14**	(**0**)
5	70	**8**	1069	(31)	33	528	(14)	8	165	(2)	29	660	(19)	22	**24**	(**0**)
6	70	**9**	1378	(56)	40	829	(29)	10	331	(9)	36	827	(32)	34	**37**	(**0**)
7	10	**10**	1500	(10)	47	951	(6)	11	694	(4)	42	1085	(6)	50	**52**	(**0**)
Tot/Avg	360	**6**	629	(105)	26	352	(51)	**6**	127	(15)	23	380	(58)	17	**18**	(**0**)

Table 2. Average experimental results comparing various drive encoding strategies.

Table 2 summarizes the results, comparing our four configurations EDGFUNCS, SUBNODES, HYPEDGES, and DIFCONS against the MAPF baseline across scenarios with increasing train counts ($|T|$, 2 to 7). The final row aggregates instances ($|I|$) and timeouts (×), and reports weighted averages for grounding (GRD) and total (TOT) times. The MAPF baseline scales poorly. Despite fast grounding times and relatively compact groundings (e.g., 2.49M rules for 7 trains), it struggles with complex instances, exhibiting the highest average runtimes and timeout rates, and failing with as few as four trains.

In contrast, our path-driven variants achieve substantially higher success rates and lower total solving times for larger train counts. Among the pure ASP variants utilizing the `map`-based encoding (Listing 1.11), SUBNODES performs best. Its superior performance correlates directly with its grounding size: SUBNODES generates compact programs (2.28M rules for 7 trains), successfully avoiding the severe computational overhead incurred with the EDGFUNCS and HYPEDGES variants (12.28M and 10.53M rules, respectively). Ultimately, the hybrid DIFCONS configuration (using SUBNODES-based path-finding) outperforms all others, maintaining low execution times with zero timeouts. Although it generates quite large programs (8.44M rules for 7 trains), *clingo*[DL] solves them exceptionally fast. This indicates that the structural encoding of the problem and the dedicated propagation of difference constraints are more critical to overall performance than raw grounding size.

7 Conclusion

While Flatland was originally conceived as a benchmark for reinforcement learning [10], the success of symbolic MAPF techniques in the 2020 Flatland competition highlighted the strengths of combinatorial approaches in this domain [8]. Building upon the successful application of ASP to real-world railway scheduling [1], we introduced *flaspland* as a dedicated testbed for tackling these challenges within a declarative framework.

This preliminary report provides a rigorous formalization of the Flatland environment, establishing a solid foundation for verifying the correctness of subsequent ASP encodings. We propose three distinct graph-based representations to capture the environment's topological and navigational constraints, and develop a two-phase ASP scheme that effectively decouples spatial routing from temporal scheduling. Our empirical evaluation demonstrates that this path-driven representation, particularly those employing sub-nodes and difference constraints, offer significant scalability advantages over off-the-shelf MAPF encodings. By bridging the gap between grid-based simulation and graph-based encodings, *flaspland* provides a scalable benchmark for evaluating the performance of ASP in complex, multi-agent railway scenarios.

References

1. Abels, D., Jordi, J., Ostrowski, M., Schaub, T., Toletti, A., Wanko, P.: Train scheduling with hybrid ASP. Theory and Practice of Logic Programming 21(3), 317–347 (2021).
2. Behrens, J., Kaminski, R., Schaub, T., Son, T., Švancara, J., Wanko, P.: Routing and scheduling in ASP applied to multi-agent path finding: Preliminary report. CoRR abs/2403.12153 (2024).
3. Brooks, D., Erdem, E., Erdogan, S., Minett, J., Ringe, D.: Inferring phylogenetic trees using answer set programming. Journal of Automated Reasoning 39(4), 471–511 (2007)
4. Erdem, E., Kisa, D., Öztok, U., Schüller, P.: A general formal framework for pathfinding problems with multiple agents. In: Proceedings of the Twenty-seventh National Conference on Artificial Intelligence (AAAI'13). pp. 290–296. AAAI Press (2013)
5. Erdem, E., Lifschitz, V., Wong, M.: Wire routing and satisfiability planning. In: Proceedings of the International Conference on Computational Logic. pp. 822–836. Springer (2000)
6. Gebser, M., Kaminski, R., Kaufmann, B., Ostrowski, M., Schaub, T., Wanko, P.: Theory solving made easy with clingo 5. In: Technical Communications of the Thirty-second International Conference on Logic Programming (ICLP'16). OASIcs, pp. 2:1–2:15. (2016)
7. Laurent, F., et al.: Flatland competition 2020: MAPF and MARL for efficient train coordination on a grid world. In: Proceedings of the NeurIPS'20 Competition and Demonstration Track. pp. 275–301. PMLR (2021)
8. Li, J., et al.: Scalable rail planning and replanning: Winning the 2020 flatland challenge. In: Proceedings of the Thirty-First International Conference on Automated Planning and Scheduling (ICAPS'21). pp. 477–485. AAAI Press (2021).
9. Lifschitz, V.: Answer Set Programming. Springer (2019).
10. Mohanty, S., et al.: Flatland-RL: Multi-agent reinforcement learning on trains. CoRR abs/2012.05893 (2020).
11. *flaspland*. `https://github.com/potassco/flaspland-toolkit`
12. *flaspland encodings*. `https://github.com/potassco/flaspland-encodings`
13. Stern, R., et al.: Multi-agent pathfinding: Definitions, variants, and benchmarks. In: Proceedings of the Twelfth International Symposium on Combinatorial Search (SOCS'19). pp. 151–159. AAAI Press (2019)